# Temco Platform

Temco Docs — documentation · built Nov 10, 2026

# Temco Docs — documentation

This folder documents the **Temco Docs module** (`extensions/temco-docs`): what it does, how it is built, what it assumes, and where it is going. The platform-wide design is design/temco-learn/.

The product documentation itself is not here: the pages and the registry live in the product repositories, because that is where the content has to be.

| Document | Read it when |
|---|---|
| `architecture.md` | you need the whole picture: the repositories, the flow, how it ships |
| `registry-schema.md` | you touch anything that reads or writes registry data |
| `extension-design.md` | you add a view, a command, a check or a diagnostic |
| `platform-design.md` | you work on the platform itself: sources, the documents a person creates, builds and artifacts, the AI over history, and the catalog |
| `editor-design.md` | you work on editing: the toolbar, creating a page, images, drafts, and the write rule they wait on |
| `app-flow.md` | you work on the left toolbar, a panel, or where a row leads: the four containers, what each view asks, and the page behind every row |
| `authoring-flow.md` | you work on screenshots, the side bar, or when a page must be re-illustrated: the capture stage, the two views, and what still waits on a decision |
| `app-layout.md` | you need the target UI: the application window, the site's generated dashboard, and the seam between them |
| `design/temco-learn/` | you need the platform design itself: content model, registry, search, the offline CHM, editing, feedback, AI, governance |

## The three names, and why they are different

| Name | What it is |
|---|---|
| **Temco Platform** | the application: the branded build of VS Code, with these modules built in |
| **Temco Docs** | this module — the documentation features, in the application and in an ordinary VS Code window |
| **Temco Learn** | the published site: the documentation people read |

One name for the tool, one for the module, one for the artifact. The module maintains content; Temco Learn displays it. Keeping them apart is what stops "put it in Temco Learn" from being ambiguous.

## The guide, and why a module has one

`temcoDocs.overview` opens **Documentation** ( `src/overview.ts` ): the tree answers "where is this page", and this page answers "what am I looking at" — pages, areas, how much is behind its source, which faults the checks found, and the three checks that can be run from here. Every number is computed from the same registry the tree reads, so the page and the tree cannot disagree. It is also the **one owner of per-area coverage**: a *Coverage & Health* page used to exist on the workbench dashboard, drawn from this same registry, and was removed rather than left to drift.

**How To Use** was removed: it was a mock guide whose numbers were typed in ("13.0 is current", "17 pages are waiting"), and a guide that quotes stale figures teaches the wrong thing.

The rest of the container is the mock's Documentation module, page for page: **Projects**, a repository, **Sources**, **Documents** and a page, a **Source Explorer**, **Review Center**, **Build Center** and **Releases** ( `src/docsPages.ts` ). The side bar holds the mock's sections and rows — Pages, Projects, Sources, Areas, Pipeline, Working Set — plus the page tree itself, which is why there is no separate *Docs Explorer* activity-bar icon: one icon browses one object.

**The Review Center** is a queue and the draft it opens: a vertical tab list on the left (each row the draft's name, the id it proposes, how long it has waited, and the source it read), and on the right exactly one draft in full — its kind, its line count, the text exactly as it will be written, and the two acts that settle it. Accept and Discard live in the draft rather than on the queue's rows, because an act that writes a page belongs beside the text it writes and a button on a row invites settling a draft nobody has read; the queue's rows are a tab list, so a keyboard reaches them the way it reaches the tabs on Publish & Releases. Only the newest is marked `Latest` , and choosing another row is what makes it the one on the right.

**The Build Center** is two answers, in the order a person needs them. **What this project builds** — a card per output the project was created with (Website, PDF, Offline help, Search index, AI index), drawn in a row like the create form's own ticks: what the output is for, the script it would run *here*, the files it has

now, and the sentence saying why when this repository has no script for it. Only a card with a script carries a **Build** button, and the card itself is never the button — reading a target must not start a build, and a target that cannot be made is reported rather than approximated with a different one. **Builds** — the runs, newest first, as a record rather than as a stack of cards: one list, a hairline between two runs, `#12`, `Latest`, when it started, how long it took counted from the two timestamps, and the verdict as a word. It folds after five runs. Above them, four figures state the project at a glance (builds · files · written · last build), and the files themselves are not reprinted here: they are listed on **Publish & Releases**, one block per output, with the fold a hundred and twenty pages need. A build history is a sequence, and a box around every run says each one is an unrelated object.

The title block carries **only the two acts this page owns** — `Build the Site` and `Build the Offline Help` — plus the per-output `Build` buttons on the rows below them ( `temcoDocs.buildOutput` ). `Refresh` was removed (reopening a page re-reads it) and `Publish & Releases` became a link in the files section it belongs to: three of four buttons in a title block were navigation, which is what made the page read as a menu rather than a record.

Every one of them is drawn with the shared page vocabulary from `temco-ui` ( `page.css` + `page()` ), so they are the pages whose look is the mock's design rather than a private `<style>` block. The pattern they establish is the one the module's other pages move to: a page describes what it draws, and `temco-ui` decides how that looks and how a click becomes a command.

**Publish & Releases** reads the same list of outputs, in the same order, and shows them as the tabs of "The files a reader can be given": a row of tabs (built, with its file count and size; or the line saying what makes it, since the search index is written by the site when a build is imported; or dashed and greyed when the create form did not tick it), and under the row exactly one panel — the files of the chosen output, the first six and a native **Show N more files** / **Show less** fold, or the callout saying why there is nothing to list. Choosing a tab is presentation only, answered by a small script the page carries: every panel is rendered, the first is open, and a window with scripts off still shows the files. The tabs are a real tab list — `aria-selected`, a roving `tabindex`, Arrow/Home/End — and the categories are never derived from whatever happens to be in the artifacts table: a target a person asked for is shown whether or not a build has produced it yet.

## Where the platform design lives

`temco-learn/design/` — 17 documents covering the content model, the registry, search, the offline CHM, the editing and git flow, feedback, the AI features, the implementation plan and the governance rules. `16-overview.md` is the one-page version.

`design/authoring-and-msdn-output.md` is the other half: what an author gets, and what a reader gets. Its §5 is the **MSDN contract** — the concrete meaning of "looks like Microsoft Learn" that a surface is judged against — and §6 is the project flow that follows from it.

**They describe the platform, and their paths belong to the product repository.** `docs/t3000/**`, `docs/help/registry.json`, `docs/docfx.json`, `scripts/docs-*.mjs` and `dist/docs/` are the

*product* repo's paths, not this one's: the content and the publishing pipeline stay there by design.

Where the two disagree, the design package wins — and the disagreement is a bug worth fixing in one place or the other.